

ONEDRAFT

CAD-AI — Aria Powered

Multi-Tenant Organization & Project Isolation Architecture

Version 1.0

Document: OneDraft Multi-Tenant Organization & Project Isolation Architecture Specification

System: OneDraft CAD-AI

Intelligence Layer: Aria

Document Class: Architectural & Implementation Stack Specification

Status: Architectural Specification

Predecessor: Billing, Subscription & Entitlement Architecture Specification

Successor: API Gateway, Rate Limiting & Service Protection Architecture Specification

1. PURPOSE

The Multi-Tenant Organization & Project Isolation Architecture establishes the formal tenancy model governing organizations, users, projects, resources, data boundaries and service access throughout OneDraft.

The purpose of this subsystem is to ensure that every organizational resource operates within an explicit and enforceable tenant boundary.

The architecture shall prevent unauthorized cross-organization access while allowing authorized users to collaborate within their assigned organization and projects.

Tenant isolation shall apply consistently across application services, databases, object storage, queues, workers, events, APIs, AI context, artifacts, observability and administrative operations.

2. ARCHITECTURAL ROLE

The Multi-Tenant Organization & Project Isolation subsystem forms the organizational isolation boundary of OneDraft.

It connects identity, authorization, billing, project lifecycle, data persistence, artifact management, runtime execution and API access into a consistent tenant model.

Primary responsibility: Establish and enforce organization and project isolation.

Primary inputs: Organization identity, user identity, memberships, project associations, authorization context and resource identifiers.

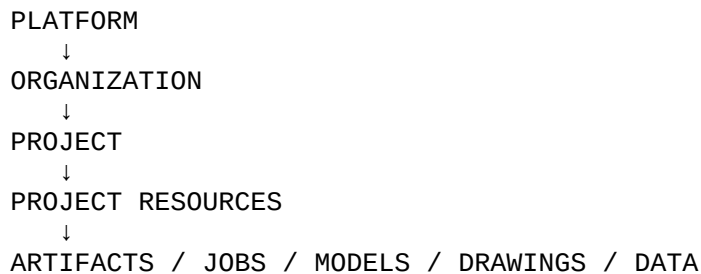
Primary outputs: Tenant context, project context, isolation decisions, resource access boundaries and tenant-scoped execution.

Authority: Organization and project ownership relationships shall be authoritative within the OneDraft domain model.

Security principle: Tenant isolation shall be enforced server-side and shall not depend upon frontend behavior.

3. TENANCY MODEL

OneDraft shall use a hierarchical tenancy model.



An organization represents the principal tenant boundary.

Projects represent controlled workspaces belonging to an organization.

Resources created within a project shall inherit the applicable organization and project context unless explicitly defined otherwise.

4. ORGANIZATION

An Organization shall represent the primary tenant entity within OneDraft.

An organization may contain:

Users

Projects

Subscriptions

Entitlements

Usage Records

Billing Records

Administrative Configuration

Artifacts

API Credentials

Integration Configuration

The organization shall provide the primary ownership boundary for these resources.

5. ORGANIZATION IDENTITY

Every organization shall possess a unique immutable identifier.

The identifier shall be used throughout the application architecture to associate resources with their tenant.

Organization identifiers shall not be reused after organization deletion or archival where such reuse could create ambiguity in historical records.

6. ORGANIZATION PROFILE

Organization metadata may include:

Organization Identifier

Organization Name

Organization Type

Status

Created Timestamp

Administrative Configuration

Subscription Reference

Billing Reference

Default Settings

Organization profile information shall remain separate from individual user identity information.

7. ORGANIZATION STATES

The canonical organization states shall include:

ACTIVE

SUSPENDED

ARCHIVED

DELETED

The applicable state shall determine whether organizational operations may proceed.

Suspension or archival shall not automatically delete organizational project data.

8. ORGANIZATION MEMBERSHIP

Users shall access organization resources through explicit membership.

Membership shall identify:

User

Organization

Role

Status

Granted Permissions

Created Timestamp

Membership Source

A user shall not gain access to an organization merely by knowing an organization or project identifier.

9. MEMBERSHIP STATES

Membership may exist in states including:

INVITED

ACTIVE

SUSPENDED

REMOVED

Membership state shall be evaluated before organization resources are accessed.

Removed membership shall terminate ordinary organization access while preserving historical records required for auditability.

10. ORGANIZATION ROLES

Organizations shall support controlled roles.

Representative roles include:

Organization Owner

Organization Administrator

Project Administrator

Project Member

Reviewer

Read-Only User

Billing Administrator

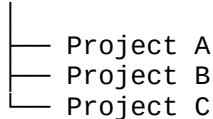
The exact role model shall remain governed by the Identity, Access & Authorization architecture.

11. PROJECT OWNERSHIP

Every project shall belong to exactly one organization.

The Project entity shall therefore contain an authoritative organization reference.

Organization



Project ownership shall not be inferred from frontend navigation or user-selected context.

12. PROJECT ISOLATION

Projects shall provide an additional logical isolation boundary within an organization.

A user may have access to one project while being denied access to another project within the same organization.

Project authorization shall therefore be evaluated independently from organization membership.

13. PROJECT MEMBERSHIP

Projects may have explicit project memberships where project-specific access is required.

Project membership shall identify:

Project

User

Role

Permissions

Status

Granted By

Timestamp

Project membership shall be subordinate to organization membership.

14. TENANT CONTEXT

Every authenticated request that accesses tenant resources shall establish a tenant context.

The tenant context shall include, as applicable:

Organization Identifier

User Identifier

Project Identifier

Membership

Role

Permissions

Subscription State

Entitlements

The context shall be established by trusted backend services.

15. CONTEXT PROPAGATION

Tenant context shall propagate through all applicable application layers.

```
API
↓
AUTHENTICATION
↓
TENANT CONTEXT
↓
AUTHORIZATION
↓
APPLICATION SERVICE
↓
DOMAIN SERVICE
↓
REPOSITORY
↓
DATABASE
```

Background workers and asynchronous jobs shall receive equivalent tenant context through trusted job metadata.

16. DATABASE ISOLATION

The PostgreSQL persistence architecture shall enforce organization and project scoping.

Tenant-aware entities shall include the applicable organization identifier.

Project-owned entities shall include or deterministically derive their project context.

Repository queries shall require tenant-aware filtering.

The system shall not depend solely upon application developers remembering to add tenant predicates to every query.

17. DATABASE CONSTRAINTS

Where appropriate, database constraints shall reinforce tenant relationships.

Examples include:

Project.organization_id → Organization.id

ProjectMember.project_id → Project.id

ProjectResource.project_id → Project.id

Composite uniqueness constraints shall include organization or project scope where required.

Tenant relationships shall be validated at persistence boundaries.

18. ROW-LEVEL ISOLATION

Where appropriate, PostgreSQL row-level security or equivalent database enforcement may be used to provide an additional tenant-isolation layer.

Database-level isolation shall complement rather than replace application-level authorization.

The architecture shall maintain defense in depth.

19. REPOSITORY ISOLATION

Repositories shall provide tenant-aware data access.

A repository operation shall execute within an explicit tenant context.

The repository layer shall prevent accidental unrestricted access to tenant-owned resources.

Administrative cross-tenant operations shall require explicit privileged execution paths.

20. OBJECT STORAGE ISOLATION

Object storage shall use tenant-aware namespaces.

A logical structure may follow:

```
organizations/  
  {organization_id}/  
    projects/  
      {project_id}/  
        artifacts/  
        imports/  
        exports/  
        drawings/  
        models/
```

Object identifiers shall not expose authorization authority.

Access to an object shall require an authorization decision independent of possession of the storage key.

21. ARTIFACT ISOLATION

Generated and imported artifacts shall remain associated with their organization and project.

Artifact metadata shall contain sufficient ownership information to establish the correct tenant context.

Artifact retrieval shall verify authorization before providing access.

22. JOB ISOLATION

Asynchronous jobs shall contain tenant and project context.

A job record shall identify:

Organization

Project, where applicable.

Initiating User

Job Type

Authorization Context

Input References

Output References

Workers shall validate tenant context before processing the job.

23. WORKER ISOLATION

Workers shall not process tenant resources solely from untrusted identifiers.

A worker shall retrieve the authoritative job record and verify its organization and project context before executing the operation.

Worker execution shall preserve tenant context when accessing databases, artifacts, model data and external integrations.

24. EVENT ISOLATION

Internal events shall include tenant context where the event concerns tenant-owned resources.

An event may contain:

Organization Identifier

Project Identifier

Resource Identifier

Event Type

Event Version

Actor

Correlation Identifier

Consumers shall validate the applicable tenant context before processing tenant resources.

25. EVENT BUS SECURITY

The event bus shall prevent unauthorized consumers from receiving or processing tenant-sensitive events.

Topic or routing structures may use organization or resource scopes where appropriate.

Administrative system events shall remain distinguishable from ordinary tenant events.

26. API TENANT BOUNDARIES

API requests shall be evaluated against tenant context.

The API shall not treat a resource identifier as sufficient proof of access.

For example:

GET /projects/{project_id}

shall resolve the project and verify that the requesting principal is authorized to access the project within its organization.

27. RESOURCE AUTHORIZATION

Every tenant-owned resource shall have an identifiable ownership path.

A resource shall be resolvable through:

Resource
↓
Project
↓
Organization

or, for organization-level resources:

Resource
↓
Organization

Authorization shall evaluate this ownership chain.

28. CROSS-TENANT ACCESS PREVENTION

Cross-tenant access shall be explicitly denied by default.

A user associated with Organization A shall not be able to retrieve or modify resources belonging to Organization B.

This restriction shall apply even when:

Resource IDs are known

URLs are known

API requests are manually constructed

Frontend controls are bypassed

Cached identifiers are retained

29. ADMINISTRATIVE ACCESS

Privileged platform administrators may require controlled cross-tenant capabilities for operational purposes.

Such access shall use explicit administrative authorization.

Administrative access shall be:

Authorized

Scoped

Audited

Time-Bounded where appropriate

Traceable

Administrative privilege shall not silently become ordinary organization membership.

30. SUPPORT ACCESS

If OneDraft provides support personnel access to customer environments, support access shall use a dedicated controlled mechanism.

Support access shall identify:

Support Actor

Organization

Project

Purpose

Authorization

Start Time

End Time

Support access shall produce an audit record.

31. BILLING ISOLATION

Billing information shall remain organization-scoped.

A project's billing impact may be recorded through usage references, but commercial authority shall remain associated with the organization subscription.

Users shall not access another organization's billing information.

Billing administrators shall receive only the commercial access permitted by their role.

32. ENTITLEMENT ISOLATION

Entitlements shall be evaluated within organization context.

The system shall determine feature availability using the correct organization subscription and applicable project context.

One organization's subscription shall not grant capabilities to another organization.

33. ARIA TENANT ISOLATION

Aria shall operate within explicit tenant and project context.

Aria context construction shall ensure that information from one organization cannot become context for another organization.

Project-specific Aria interactions shall be limited to the authorized project's information.

Aria shall not infer authorization merely because information exists within the platform.

34. AI CONTEXT BOUNDARY

AI context construction shall enforce:

Organization Boundary

Project Boundary

User Authorization

Data Classification

Resource Authorization

Context Scope

Only information authorized for the current interaction shall be included in the AI context.

35. AI MEMORY ISOLATION

Persistent AI memory shall be explicitly scoped.

Memory may be:

User-Scoped

Organization-Scoped

Project-Scoped

System-Scoped

The system shall not permit project or organization information to be unintentionally promoted into a broader memory scope.

36. SEARCH ISOLATION

Search operations shall be tenant-aware.

Search indexes shall maintain organization and project metadata.

Search queries shall be filtered according to the user's authorized scope before results are returned.
Search ranking shall not cause unauthorized records to become visible.

37. CACHE ISOLATION

Caches containing tenant-sensitive information shall use tenant-aware keys.

Example:

```
organization:{organization_id}:project:{project_id}:resource:{resource_id}
```

Shared caches shall never return tenant-sensitive objects solely by an unscoped resource identifier.

Cache invalidation shall preserve tenant boundaries.

38. SESSION ISOLATION

Authenticated sessions shall maintain the user's authorized organization context.

Where a user belongs to multiple organizations, the active organization shall be explicitly established.

Switching organizations shall cause tenant-scoped application state to be refreshed.

Previously selected project data shall not remain visible after an unauthorized tenant switch.

39. FRONTEND TENANT CONTEXT

The frontend shall clearly identify the active organization and project.

Organization switching shall be explicit.

Project navigation shall operate only within the selected organization.

Frontend state shall be cleared or reconciled when organization context changes.

Backend authorization remains authoritative regardless of frontend state.

40. DATA EXPORT ISOLATION

Exports shall remain tenant-scoped.

An export job shall include organization and project context.

Generated export artifacts shall inherit the applicable tenant ownership.

Cross-organization export operations shall require explicit administrative authority where supported.

41. IMPORT ISOLATION

Imported data shall be associated with the organization and project initiating the import.

External source files shall not become accessible to another organization unless explicitly transferred through an authorized mechanism.

Import processing jobs shall preserve the originating tenant context.

42. INTEGRATION ISOLATION

External integrations shall maintain tenant-specific credentials and configuration.

Integration records shall identify their owning organization.

External service calls shall use the credentials and configuration associated with the authorized tenant.

Shared platform integrations shall not expose one organization's credentials or project information to another organization.

43. OBSERVABILITY ISOLATION

Tenant-sensitive telemetry shall be appropriately scoped.

Logs, traces and metrics shall contain tenant identifiers where required for operational diagnosis while minimizing unnecessary exposure of customer data.

Users shall not gain access to another organization's operational telemetry.

Platform operators may access broader telemetry through controlled administrative permissions.

44. BACKUP ISOLATION

Backups shall preserve tenant ownership relationships.

Restore operations shall maintain organization and project associations.

A tenant-specific restore shall not accidentally overwrite or expose another organization's resources.

Disaster-recovery procedures shall be tested for tenant-boundary integrity.

45. DATA LIFECYCLE

Tenant ownership shall remain preserved throughout the data lifecycle.

Retention, archival and deletion operations shall respect organization and project boundaries.

Deleting or archiving one organization's data shall not affect another organization's resources.

Historical records shall retain sufficient ownership metadata to preserve provenance.

46. ORGANIZATION TRANSFER

OneDraft may support controlled transfer of a project between organizations where product policy permits.

A transfer shall require explicit authorization from the applicable organizations.

The operation shall establish:

Source Organization

Destination Organization

Project

Authorized Actors

Effective Time

Transfer Record

Updated Ownership

Historical ownership shall remain preserved.

47. PROJECT CLONING

Project cloning shall create a new project identity.

A clone shall not inherit unrestricted access to the source project's authorization context.

The new project shall belong to the explicitly selected destination organization.

Copied resources shall receive appropriate new ownership references.

48. TENANT-AWARE API & DATA MODEL

Representative tenant-aware entities include:

Organization

OrganizationMember

Project

ProjectMember

TenantContext

TenantResource

TenantAccessGrant

OrganizationConfiguration

ProjectOwnershipRecord

OrganizationTransfer

SupportAccessSession

TenantAuditRecord

These entities shall integrate with the existing User, Project, Subscription, Entitlement, ModelVersion, Artifact, Job, Event and Authorization structures.

49. ARCHITECTURAL INVARIANTS

The following invariants shall govern the Multi-Tenant Organization & Project Isolation Architecture.

Single Ownership: Every project shall belong to one organization at a given point in time.

Default Denial: Cross-tenant access shall be denied unless explicitly authorized.

Backend Enforcement: Tenant isolation shall not depend upon frontend behavior.

Context Integrity: Tenant context shall be preserved across synchronous and asynchronous execution.

Data Isolation: Databases, artifacts, caches, events, jobs and AI context shall preserve tenant boundaries.

Authorization: Resource identifiers shall never constitute authorization.

AI Isolation: Aria shall not receive unauthorized cross-tenant context.

Billing Isolation: Commercial information shall remain organization-scoped.

Administrative Control: Cross-tenant administrative operations shall require explicit privileged authorization.

Historical Integrity: Ownership changes shall preserve historical records.

No Silent Transfer: Projects shall not change organizational ownership without an explicit controlled operation.

Defense in Depth: Application, repository, database and infrastructure controls shall reinforce one another.

50. COMPLETION STATUS

The Multi-Tenant Organization & Project Isolation Architecture establishes the complete organizational and project isolation model for OneDraft.

The subsystem defines organization ownership, memberships, project boundaries, tenant context, database isolation, object-storage isolation, job and worker isolation, event isolation, API boundaries, billing and entitlement isolation, Aria context isolation, search and cache isolation, integration boundaries, observability, backup, data lifecycle, organization transfers and project cloning.

The architecture establishes the tenant boundary required for OneDraft to operate as a secure multi-organization platform while preserving the integrity and independence of project data and services.

The next architectural layer establishes the API Gateway, Rate Limiting & Service Protection architecture, defining how external and internal requests are admitted, authenticated, authorized, throttled, protected and routed through the OneDraft platform.

Status: ARCHITECTURALLY DEFINED

Next Document: API Gateway, Rate Limiting & Service Protection Architecture Specification

Overall Architecture: OneDraft End-to-End System Integration